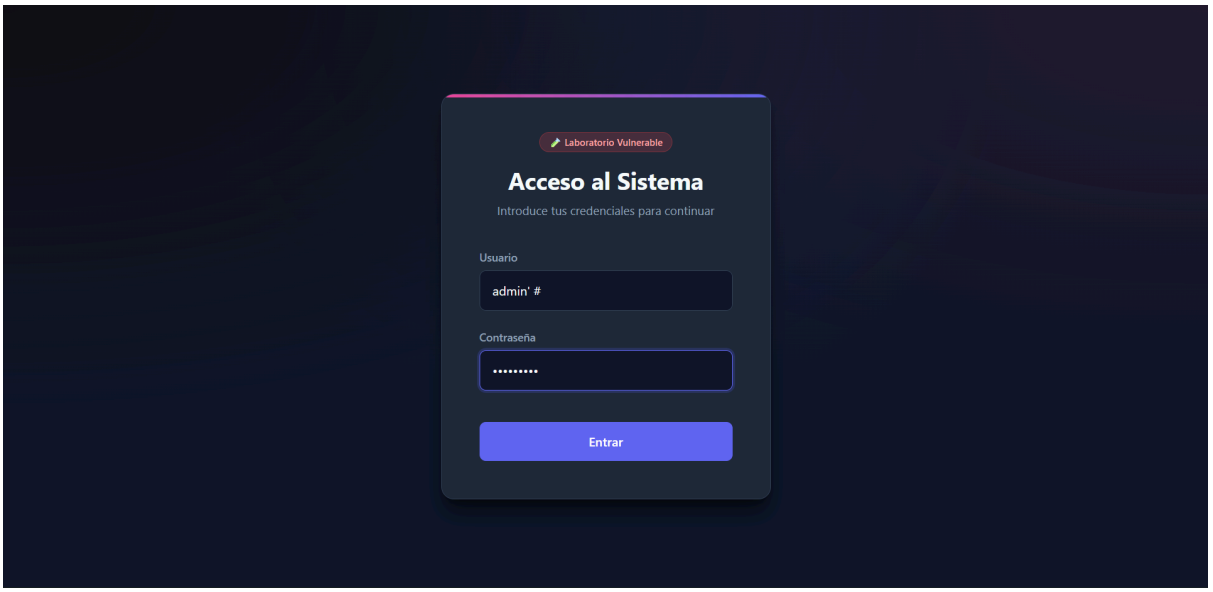
SQL Injection en PHP

Ejercicio 1 · Laboratorio vulnerable



El símbolo **#** en MariaDB/MySQL sirve para comentar el resto de la consulta. La SQL real ejecutada quedará así: SELECT * FROM usuarios WHERE username = 'admin' #' AND password = '<contraseña_aleatoria>'.

Ejercicio 2 · UNION SELECT — Volcar usuarios

Ejercicio 3 · Listar bases de datos

Ejercicio 4 · Versión segura

- **Imposibilita Ingeniería Inversa:**
  password_hash() utiliza el algoritmo bcrypt, creando un hash unidireccional acompañado de una salt matemática aleatoria y un control de retraso (work factor). Esto impide que la base de datos contenga las contraseñas reales.

- **Mantiene privacidad ante filtraciones (Dump):**
  Si un atacante consiguiera hacer un dumpeo de la base de datos por un error en otra parte de la aplicación web, lo único que vería en la columna de contraseñas sería ruido en forma de hash (ej. $2y$10$Ua...), por lo que seguiría sin saber tus claves e imposibilitaría el uso de tablas rainbow (fuerza bruta precalculada) para desencriptarlas.

- **Validación en Backend Seguro:**
  password_verify() es el único que sabe comparar matemáticamente el texto que ha puesto tu intento de conexión ($password_input) con el hash encriptado de la Base de Datos. Solo dará OK si operativamente devuelven el mismo resultado, imposibilitando inyectar nada en este nivel aplicativo.